NEXTRY

Guía de comandos .nxf

Referencia práctica en español · 1 de noviembre de 2026

Versión del compilador de NXK, con las nuevas órdenes romper y continuar. Esta guía describe capacidades implementadas; no es una especificación de funciones futuras.

Lenguaje pequeño para aprender a escribir módulos del núcleo y aplicaciones nativas de NXK. Está implementado: su compilador analiza la fuente, comprueba los tipos y genera C, que Clang/LLD convierte en instrucciones del procesador. La sintaxis no se ejecuta como Python dentro del kernel.

El compilador inicial está escrito en Python y trae su intérprete portátil como herramienta de construcción en Windows. Las aplicaciones compiladas no necesitan ese intérprete.

Nuestros archivos

Extensión · Significado

`.nxf` · Fuente editable en NXK Español, guardada como UTF-8.

`.nxe` · Ejecutable binario nativo de NXK, con cabecera NXE propia.

`.exe` · Ejecutable de Windows; lo atiende el puente de compatibilidad, no el cargador NXE.

El archivo `kernel/inicio.nxf` se integra directamente al núcleo. El archivo `lenguaje/ejemplos/hola.nxf` se compila como aplicación separada `build/x86/hola.nxe`.

Primer programa

```
función inicio
    mostrar "Hola desde mi aplicacion"
    variable contador = 0
    repetir 3 veces
        contador = contador + 1
    fin
    número contador
fin
```

`función` también puede escribirse `funcion`, y `número` puede escribirse `numero`. Las dos formas significan lo mismo.

Reglas básicas

1. Una orden por línea. No se escriben puntos y coma. 2. Los bloques terminan con `fin`; la sangría solo mejora la lectura. 3. Cada archivo tiene exactamente una función `inicio`. Puede declarar otras funciones. 4. Las palabras clave se escriben en minúsculas y distinguen mayúsculas de minúsculas. 5. Los nombres empiezan con una letra o _; después admiten letras, números y _. Se permiten ñ y tildes. No se pueden usar palabras reservadas como nombres. 6. Los comentarios empiezan con # y terminan al finalizar la línea. 7. Toda variable y constante se crea con un valor inicial. 8. El tipo se deduce del primer valor y se mantiene. No hay conversiones automáticas entre texto, entero y lógico. 9. Una variable pertenece al bloque donde se declara. No se puede usar después de salir de ese bloque ni redeclarar un nombre que siga visible. 10. Las funciones de esta versión no reciben argumentos ni devuelven valores. `volver` termina la función actual. Las llamadas recursivas se rechazan.

Valores

Tipo · Ejemplos · Comportamiento

Entero · `0`, `42`, `0x3fd` · Sin signo, de 32 bits: de 0 a 4294967295. Igual en NXK x86 y x64.

Texto · `"Hola"` · Inmutable, entre comillas dobles. Admite `\n`, `\t`, `\"`, `\\` y escapes Unicode `\uXXXX`.

Lógico · `verdadero`, `falso` · Resultado de comparaciones y operaciones lógicas.

La suma, resta y multiplicación de enteros desbordan módulo 2^{32}: por ejemplo, `4294967295 + 1` produce cero y `-1` produce 4294967295. No hay números negativos con signo ni decimales en esta versión.

La división entera descarta el resto. Dividir o calcular resto con divisor cero detiene el programa con un mensaje. Un divisor cero literal se rechaza al compilar.

Los textos se guardan en UTF-8. La salida serie conserva esos bytes; el controlador VGA actual todavía no representa correctamente todos los caracteres Unicode. Para los mensajes de las primeras pruebas se emplea texto ASCII.

Palabras clave y órdenes

Palabra · Uso

`funcion` / `función` · Comenzar una función.

`fin` · Cerrar un bloque.

`variable` · Declarar un valor modificable: `variable cuenta = 0`.

`constante` · Declarar un valor que no se puede reasignar. Su valor inicial puede calcularse al ejecutar.

`si`, `sino` · Elegir entre bloques según una condición lógica.

`mientras` · Repetir mientras una condición lógica sea verdadera.

`repetir`, `veces` · Ejecutar un bloque una cantidad entera de veces.

`mostrar` · Escribir un texto y un salto de línea.

`numero` / `número` · Escribir un entero y un salto de línea.

`llamar` · Invocar una función sin argumentos: `llamar bienvenida`.

`volver` · Salir de la función actual.

`afirmar` · Exigir una condición lógica; informa la línea y detiene la ejecución si falla.

`detener` · Detener la CPU. En este prototipo privilegiado también detiene el kernel.

`verdadero`, `falso` · Valores lógicos.

`y`, `o`, `no` · Operaciones lógicas. `y` y `o` evalúan con cortocircuito.

`y_bits`, `o_bits`, `xor_bits` · Operaciones bit a bit entre enteros.

`memoria_usada()` · Bytes usados por el pequeño asignador del kernel, no toda la RAM del equipo.

`leer_puerto` · Leer un byte: `leer_puerto estado, 0x3fd`. El destino debe ser una variable entera.

`escribir_puerto` · Escribir un byte: `escribir_puerto 0xe9, 65`.

Los puertos deben estar entre 0 y 65535 y el byte de salida entre 0 y 255; el puente comprueba esos límites. Operar un puerto válido modifica el hardware: estas instrucciones son primitivas privilegiadas para desarrollar controladores.

Operadores y orden de evaluación

De menor a mayor prioridad:

1. o 2. y 3. ==, != 4. <, >, <=, >= 5. o_bits 6. xor_bits 7. y_bits 8. <<, >> 9. +, - 10. *, /, % 11. no y el menos unario

Se pueden usar paréntesis para expresar la intención. Escribe no (contador == 3) para negar una comparación. Las comparaciones encadenadas no están admitidas: escribe a < b y b < c. La igualdad de textos no está implementada.

Los desplazamientos usan los cinco bits bajos de la cantidad: 1 << 33 equivale a 1 << 1. Los enteros no se convierten implícitamente en condiciones: escribe si cuenta != 0.

repetir cantidad veces calcula la cantidad una sola vez al entrar. mientras comprueba su condición en cada vuelta. Un bucle infinito puede bloquear el prototipo; todavía no hay un planificador que lo interrumpa.

Ejemplos de bloques

```
funcion saludo
    mostrar "Bienvenido a NXK"
fin

funcion inicio
    llamar saludo
    constante limite = 3
    variable cuenta = 0
    mientras cuenta < limite
        numero cuenta
        cuenta = cuenta + 1
    fin
    si cuenta == limite
        mostrar "Terminado"
    sino
        mostrar "Resultado inesperado"
    fin
    afirmar cuenta == 3
fin
```

Construir y probar

• Ejecuta CREAR-APP-NXE.cmd para crear build/x86/hola.nxe a partir del ejemplo.

• Ejecuta COMPILAR-NXK.cmd para reconstruir el núcleo e incluir esa aplicación de prueba.

• Cierra la ventana anterior y abre INICIAR-NXK-32.cmd.

• En NXK escribe ejecutar hola.nxe y pulsa Enter.

• PROBAR-LENGUAJE.cmd comprueba el compilador. PROBAR-NXK-32.cmd comprueba también el módulo del lenguaje, la aplicación NXE y el EXE de prueba dentro del emulador.

La versión inicial solo expone los archivos de ejemplo embebidos durante la construcción. El shell todavía no carga rutas arbitrarias desde un sistema de archivos.

Alcance de la versión inicial

Esta sintaxis reduce conceptos para nuestras primeras tareas de kernel; que resulte más sencilla que Python depende de la tarea y de quien la use. La implementación actual cubre las reglas de este documento. Estructuras, arrays, parámetros, retorno de valores, módulos importables, gestión de memoria avanzada y compilación directa a código máquina quedan para versiones posteriores.

El código generado conserva una pequeña base C y ensamblador para el arranque y el acceso al procesador. Las aplicaciones NXE actuales se ejecutan con privilegios del kernel, una a la vez, sin aislamiento de procesos. No constituyen todavía una plataforma para ejecutar programas externos desconocidos.

Control de bucles

`romper` sale del bucle mas cercano. `continuar` salta a su siguiente vuelta. Solo se admiten dentro de `mientras` o `repetir`, incluso anidados en `si`. No atraviesan llamadas a funciones. Ejemplo: lenguaje/ejemplos/bucles.nxf.

Nuevo ejemplo: controlar un bucle

```
# Control de bucles: salta 2 y termina al llegar a 5.
funcion inicio
    variable cuenta = 0
    variable suma = 0
    mientras verdadero
        cuenta = cuenta + 1
        si cuenta == 2
            continuar
        fin
        si cuenta == 5
            romper
        fin
        suma = suma + cuenta
    fin
    afirmar suma == 8
    numero suma
fin
```

Crear tus aplicaciones

Abre NXK en Visual Studio Code. Crea mi_programa.nxf en la raíz del proyecto y usa el siguiente comando desde su terminal:

```
.\compilar-app.ps1 -Fuente mi_programa.nxf
  -Salida build/x86/mi_programa.nxe
```

Escribe el comando anterior en una sola línea. Genera un NXE para NXK; no es un EXE de Windows. El shell actual ejecuta la aplicación hola.nxe integrada al construir. Todavía no carga aplicaciones arbitrarias desde los archivos RAM.

Diferencia entre comandos

mostrar, variable y repetir pertenecen al lenguaje .nxf. help, mem, tareas, escribir y leer son comandos de la consola de NXK; no son instrucciones NEXTRY. Las fuentes .nxa usan NEXTRY ASM y quedan fuera de esta guía.